

■ 강의명: CSCI103

■ 주차: Lecture 14

■ 교수명: UIUC Student

■ 목적: 이 문서는 CSCI103 Lecture 14 강의 내용을 바탕으로 작성된 종합 학습 노트입니다. 데이터브릭스(Databricks) 플랫폼의 최신 기능과 로드맵, 그리고 데이터 인텔리전스, 거버넌스, AI 에이전트 구축, 데이터 웨어하우징 및 엔지니어링 분야의 주요 투자 영역을 다룹니다. 특히 Lakebase, Agent Bricks, AI Parse Document, Databricks One, ABAC 등 새롭게 출시되거나 출시 예정인 기능들을 비전공자도 이해하기 쉽게 설명합니다. 또한, 퀴즈 2와 과제 3의 상세 해설을 포함하여 핵심 개념과 문제 해결 방법을 명확히 제시합니다. 이 문서는 시험 대비 및 데이터브릭스 플랫폼 이해를 위한 독립적인 학습 자료로 활용될 수 있도록 구성되었습니다.

Contents

용어 정리

다음 표는 강의에서 다루는 주요 용어들을 비전공자도 이해하기 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
데이터브릭스 (Databricks)	데이터 분석, AI 모델 개발, 데이터 웨어하우스를 위한 통합 플랫폼	Databricks	레이크하우스 아키텍처 기반
레이크하우스 (Lakehouse)	데이터 레이크의 유연성과 데이터 웨어하우스의 구조적 장점을 결합한 아키텍처	Lakehouse	데이터브릭스의 핵심
OLTP	실시간 트랜잭션 처리 (예: 은행 거래)	Online Transaction Processing	Lakebase가 이 역할을 수행
OLAP	대규모 데이터 분석 및 보고 (예: 비즈니스 인텔리전스)	Online Analytical Processing	데이터브릭스의 전통적 강점
델타 레이크 (Delta Lake)	데이터 레이크에 ACID 트랜잭션, 스키마 적용, 시간 여행 기능을 제공하는 오픈 소스 저장소 계층	Delta Lake	데이터 일관성 및 신뢰성 확보
아이스버그 (Iceberg)	대규모 분석 데이터셋을 위한 오픈 테이블 형식	Iceberg	델타 레이크와 유사한 역할
레이크베이스 (Lakebase)	데이터브릭스에서 제공하는 트랜잭션 데이터베이스 (PostgreSQL 기반)	Lakebase	OLTP 워크로드 지원
에이전트 브릭스 (Agent Bricks)	AI 에이전트를 쉽게 구축하고 배포할 수 있도록 돕는 도구	Agent Bricks	AutoML과 유사한 방식
AIBI	데이터브릭스에서 제공하는 비즈니스 인텔리전스 및 대시보드 기능	AIBI (AI-powered BI)	대시보드와 지니(Genie) 포함
레이크플로우 (LakeFlow)	데이터 수집, ETL, 스트리밍 처리를 위한 데이터 엔지니어링 도구	LakeFlow	데이터 파이프라인 구축
앱스 (Apps)	데이터브릭스 플랫폼 위에 구축되는 사용자 친화적인 애플리케이션	Apps	대시보드보다 양방향 소통 가능
마켓플레이스 (Marketplace)	데이터, 모델, 코드를 발견하고 공유하는 플랫폼	Marketplace	내부/외부 공유 가능
ABAC	속성 기반 접근 제어. 데이터의 특정 속성 (예: PII)에 따라 접근 권한을 부여	Attribute-Based Access Control	RBAC보다 유연하고 강력
RBAC	역할 기반 접근 제어. 사용자 역할에 따라 접근 권한을 부여	Role-Based Access Control	전통적인 접근 제어 방식
MCP 서버	모델, 코드, 데이터 등을 포함하는 사용자 정의 서버	MCP Servers	AI 에이전트가 참조 가능
AI Parse Document	SQL 함수를 사용하여 PDF 등 문서에서 구조화된 데이터를 추출하는 기능	AI Parse Document	비정형 데이터를 정형화
서버리스 GPU 컴퓨트	GPU 자원을 필요할 때만 사용하고 사용량에 따라 비용을 지불하는 방식	Serverless GPU Compute	AI/ML 워크로드 가속화
멀티 스테이트먼트 트랜잭션	여러 SQL 문장을 하나의 논리적 단위로 묶어 모두 성공하거나 모두 실패하도록 처리	Multi-statement Transaction	데이터 일관성 유지
저장 프로시저 (Stored Procedure)	미리 정의된 SQL 코드 블록으로, 복잡한 작업을 효율적으로 수행	Stored Procedure	데이터베이스 로직 캡슐화
레이크브리지 (Lakebridge)	레거시 데이터 시스템에서 데이터브릭스로 데이터를 마이그레이션하는 도구	Lakebridge	데이터 마이그레이션 간소화
공간 SQL (Spatial SQL)	지리적 데이터(위치, 영역 등)를 처리하고 분석하는 SQL 기능	Spatial SQL	지도 기반 분석 가능
오토로더 (Autoloader)	클라우드 스토리지에서 증분적으로 데이터를 로드하는 기능	Autoloader	스트리밍 데이터 수집
체크포인팅 (Checkpointing)	스트리밍 처리 상태를 주기적으로 저장하여 장애 발생 시 복구 지점을 제공	Checkpointing	스트리밍의 내결함성 확보
아이덴포텐트 싱크 (Idempotent Sink)	동일한 작업을 여러 번 수행해도 결과가 항상 동일하도록 보장하는 데이터 저장소	Idempotent Sink	정확히 한 번 처리(Exactly-Once) 지원
브로드캐스트 변수 (Broadcast Variable)	Spark 드라이버에서 워커 노드로 읽기 전용 데이터를 효율적으로 배포하는 변수	Broadcast Variable	네트워크 오버헤드 감소
셔플 (Shuffle)	Spark에서 데이터를 재분배하여 병렬 처리를 가능하게 하는 과정	Shuffle	성능 병목의 원인이 될 수 있음
AQE (Adaptive Query Execution)	Spark SQL 쿼리 실행 계획을 런타임에 동적으로 조정하여 성능을 최적화하는 기능	Adaptive Query Execution	쿼리 성능 자동 개선
SCD (Slowly Changing Dimension)	시간이 지남에 따라 변경되는 차원 데이터를 관리하는 기법	Slowly Changing Dimension	데이터 웨어하우스에서 중요
옵투나 (Optuna)	하이퍼파라미터 최적화를 위한 오픈 소스 프레임워크	Optuna	머신러닝 모델 성능 향상
원-핫 인코딩 (One-Hot Encoding)	범주형 데이터를 이진 벡터 형태로 변환하는 기법	One-Hot Encoding	머신러닝 모델 입력 준비
앙상블 학습 (Ensemble Learning)	여러 모델을 결합하여 단일 모델보다 더 나은 예측 성능을 얻는 기법	Ensemble Learning	모델 강건성 향상
PCA (Principal Component Analysis)	고차원 데이터를 저차원으로 축소하는 통계 기법	Principal Component Analysis	차원 축소
MLflow 모델 레지스트리	머신러닝 모델의 버전 관리, 배포, 거버넌스를 위한 중앙 집중식 저장소	MLflow Model Registry	모델 수명 주기 관리

핵심 개념 및 원리

0.1 데이터브릭스 플랫폼의 진화와 레이크하우스 아키텍처

데이터브릭스는 데이터 레이크의 유연성과 데이터 웨어하우스의 구조적 장점을 결합한 레이크하우스 아키텍처를 기반으로 합니다. 이는 분석(OLAP)과 트랜잭션(OLTP) 워크로드를 모두 지원하며, 데이터 인텔리전스, 거버넌스, AI 에이전트 구축, 데이터 웨어하우징 및 엔지니어링 분야에 집중적으로 투자하고 있습니다.

레이크하우스 아키텍처의 중요성

레이크하우스는 기존의 데이터 레이크와 데이터 웨어하우스의 장점을 결합하여, 모든 데이터 유형(정형, 비정형)을 저장하고, SQL 분석부터 머신러닝까지 다양한 워크로드를 단일 플랫폼에서 처리할 수 있도록 합니다. 이는 데이터 사일로(silo)를 제거하고 데이터 관리의 복잡성을 줄여줍니다.

0.1.1 주요 투자 영역

데이터브릭스는 다음 네 가지 핵심 영역에 집중하여 플랫폼을 발전시키고 있습니다.

1. **데이터 인텔리전스 기능 (Data Intelligence Capabilities):** 플랫폼 자체를 더 스마트하게 만들어 사용자가 해야 할 작업을 줄이고 자동화를 늘립니다.
2. **내장된 데이터 거버넌스, 보안 및 공유 도구 (Built-in Data Governance, Security and Sharing Tools):** 데이터의 안전한 관리와 효율적인 공유를 위한 기능을 강화합니다.
3. **AI 에이전트 구축 도구 (Tools to Build AI Agents):** AI 에이전트 개발 및 배포를 위한 프레임워크와 환경을 제공합니다.
4. **데이터 웨어하우징, 데이터 마이그레이션 및 데이터 엔지니어링 (Warehousing, Data Migration and Data Engineering):** 데이터 파이프라인 구축, 레거시 시스템 마이그레이션, 고성능 데이터 웨어하우징을 지원합니다.

0.2 새로운 기능 및 로드맵

강의에서는 아직 완전히 안정화되지 않았지만, 공개 프리뷰(Public Preview) 또는 곧 출시될 예정인 데이터브릭스의 새로운 기능들을 소개합니다.

0.2.1 Lakebase: 트랜잭션 데이터베이스의 새로운 패러다임

Lakebase는 데이터브릭스 플랫폼에 새롭게 추가된 트랜잭션 데이터베이스(OLTP) 기능으로, PostgreSQL 구현을 기반으로 합니다.

Lakebase의 핵심 특징

Lakebase는 전통적인 관계형 데이터베이스의 기능을 제공하면서도, 클라우드 네이티브 환경에 최적화된 혁신적인 특징들을 가집니다.

- **스토리지와 컴퓨트 분리 (Storage and Compute Segregation):**
 - **개념:** 데이터 저장소(스토리지)와 데이터 처리 엔진(컴퓨트)이 독립적으로 작동합니다.
 - **장점:** 컴퓨트 자원은 워크로드에 따라 동적으로 확장(scale up)되거나 축소(scale down)될 수 있으

며, 사용하지 않을 때는 0으로 줄어들어 비용을 절감할 수 있습니다. 이는 전통적인 데이터베이스 서버가 항상 실행되어야 하는 방식과 근본적으로 다릅니다.

- **비유:** 마치 수도꼭지(컴퓨터)를 틀 때만 물(데이터 처리)이 나오고, 잠그면 물이 나오지 않지만 물탱크(스토리지)의 물은 그대로 있는 것과 같습니다.

- **브랜칭 (Branching):**

- **개념:** Git과 유사하게 데이터베이스의 여러 버전을 생성하고 관리할 수 있습니다. 기본적으로 프로덕션(Production) 브랜치와 개발(Development) 브랜치가 제공됩니다.
- **장점:** 개발팀은 프로덕션 환경에 영향을 주지 않고 안전하게 새로운 기능을 개발하고 테스트할 수 있습니다.
- **비유:** 소프트웨어 개발에서 코드를 복사하여 새로운 기능을 개발하고, 문제가 없으면 원본 코드에 병합하는 것과 같습니다.

- **스냅샷 및 시간 여행 (Snapshots and Time Travel):**

- **개념:** 특정 시점의 데이터베이스 상태를 저장하고, 필요할 때 해당 시점으로 복구하거나 과거 데이터를 조회할 수 있습니다.
- **장점:** 데이터 손실 위험을 줄이고, 데이터 변경 이력을 쉽게 추적할 수 있습니다.

- **SQL 편집기 및 모니터링 (SQL Editor and Monitoring):**

- **개념:** SQL 쿼리를 작성하고 실행할 수 있는 환경과 쿼리 성능, 시스템 운영을 모니터링하는 기능을 제공합니다.

- **카탈로그 통합 (Catalog Integration):**

- **개념:** Lakebase 데이터베이스를 데이터브릭스 유니티 카탈로그(Unity Catalog)에 연결하여 관리할 수 있습니다.
- **장점:** 데이터브릭스 플랫폼의 다른 데이터 소스와 함께 Lakebase 데이터를 통합적으로 관리하고 접근할 수 있습니다.

Lakebase 카탈로그 생성 예시

유니티 카탈로그에서 Lakebase 유형의 카탈로그를 생성하여 기존 Lakebase 데이터베이스 인스턴스와 연결할 수 있습니다.

```

1 -- 1. Databricks 에서 UI Catalog -> Create Catalog 선택
2 -- 2. 에서 Type 'Lakebase Postgress' 선택
3 -- 3. 카탈로그 이름 지정예 (: my_lakebase_catalog)
4 -- 4. Autoscaling 설정 컴퓨터 ( 자원 자동 확장 축소 /)
5 -- 5. 연결할 Lakebase 데이터베이스 인스턴스 선택
6 -- 6. 데이터베이스 이름 지정예 (: my_db)
7 -- 7. 'Create database if doesn't exist' 선택 후 생성
  
```

Listing 1: Lakebase 카탈로그 생성 과정

이렇게 생성된 카탈로그를 통해 SQL 쿼리로 Lakebase 데이터에 접근할 수 있습니다: `SELECT * FROM mylakebase_catalog.mydb.mytable;`

0.2.2 Databricks One: 비즈니스 사용자를 위한 통합 포털

Databricks One은 비즈니스 사용자가 데이터브릭스 플랫폼의 다양한 기능을 쉽게 접근하고 활용할 수 있도록 설계된 통합 포털입니다.

Databricks One의 특징

Databricks One은 대시보드, 지니(Genie), 레이크하우스 앱 등 비즈니스 관련 자산들을 한곳에 모아 사용자 경험을 단순화합니다.

- **계정 수준 접근 (Account Level Access):** 여러 워크스페이스에 분산된 자산들을 계정 전체에서 통합적으로 볼 수 있습니다.
- **사용자 정의 URL (Vanity URL):** companyxyz.com과 같이 회사 도메인에 맞는 URL을 설정할 수 있습니다.
- **도메인 통합 (Domain Integration):** 마케팅, 재무, 기술 등 비즈니스 도메인별로 대시보드, 지니 공간, 앱을 구성하여 직관적인 검색 및 탐색을 가능하게 합니다.

0.2.3 Apps: 양방향 소통이 가능한 애플리케이션

데이터브릭스 Apps는 플랫폼의 큐레이션된 데이터 위에 구축되는 사용자 친화적인 애플리케이션입니다.

Apps의 특징 및 대시보드와의 차이점

Apps는 대시보드처럼 데이터를 시각화할 뿐만 아니라, 사용자가 데이터를 직접 수정하고 플랫폼의 다른 서비스(모델, 벡터 검색 등)와 상호작용할 수 있는 양방향 기능을 제공합니다.

- **다양한 프레임워크 지원:** Dash, Flask, Gradio, Shiny (Python), NodeJS, React 등 다양한 웹 프레임워크를 사용하여 앱을 개발할 수 있습니다.
- **양방향 통신:** 대시보드가 주로 데이터를 읽는 단방향인 반면, Apps는 사용자가 데이터를 읽고(Read) 수정(Write) 할 수 있는 양방향 통신을 지원합니다. 예를 들어, 사용자가 데이터의 특정 값을 직접 수정하여 반영할 수 있습니다. **확장성:** 수평적 확장(Horizontal Scaling)을 지원하여 많은 사용자에게 안정적으로 서비스를 제공할 수 있습니다.

Apps 생성 예시

데이터브릭스 컴퓨트(Compute) 섹션에서 'Apps'를 선택하여 새로운 앱을 생성할 수 있습니다.

```

1 # Streamlit 템플릿을 사용하여 간단한 데이터 앱 생성
2 # 1. Databricks 에서 UI Compute -> Apps 선택
3 # 2. 'Create new app' 클릭
4 # 3. 템플릿으로 'Streamlit' 선택
5 # 4. 'Data App' 또는 'Hello World' 템플릿 선택
6 # 5. 앱 이름 지정 후 생성
7 # 이 앱은 파이프라인의 골드 (Gold) 계층 데이터를 시각화하고 ,
8 # 사용자가 특정 값을 수정할 수 있는 인터페이스를 제공할 수 있습니다 .

```

Listing 2: Streamlit 기반 Hello World 앱 예시

0.2.4 AIBI (AI-powered Business Intelligence)

AIBI는 대시보드와 지니(Genie) 기능을 통합하여 비즈니스 인텔리전스를 강화합니다.

- **대시보드 구독:** Slack, Teams 채널 구독, 계정 수준 사용자 구독, 다중 페이지 구독, CSV 첨부 등 다양한 보고 기능을 제공합니다.

- **지니 (Genie):**

- **개념:** 자연어 질문을 통해 데이터에 대한 사실적(factual) 답변을 얻을 수 있는 도구입니다.
- **연구 모드 (Research Mode):** '무엇(What)'에 대한 질문을 넘어 '왜(Why)'라는 가설 기반의 질문에 답할 수 있도록 돕는 기능입니다. 예를 들어, "관세가 오르면 내 포트폴리오에 어떤 영향을 미칠까?"와 같은 질문에 대해 다양한 시나리오와 사고 과정을 제시합니다.
- **문서 업로드 및 동적 지식 추출:** CSV, Excel, PDF 등 다양한 형식의 문서를 업로드하여 구조화된 데이터와 결합하여 분석할 수 있습니다. 이를 통해 비정형 문서에서 비즈니스 컨텍스트를 추출하고 통찰력을 풍부하게 합니다.

0.2.5 AI 에이전트 구축 도구

데이터브릭스는 AI 에이전트 개발을 위한 다양한 도구와 기능을 제공합니다.

- **Agent Bricks:** 정보 추출, 지식 보조, 멀티 에이전트 감독, 커스텀 LLM 생성 등 다양한 유형의 에이전트를 AutoML처럼 쉽게 구축할 수 있도록 돕습니다. 데이터셋 위치와 추출할 필드를 지정하는 것만으로 에이전트를 생성할 수 있습니다.
- **온라인 피처 스토어 (Online Feature Store):** 머신러닝 모델 학습 및 추론에 필요한 피처(특징)를 저장하고 관리하는 저장소입니다. 기존에는 델타 테이블로 백업되었으나, 이제 Lakebase로도 백업될 예정입니다.
- **AI Parse Document:** SQL 함수를 사용하여 PDF 문서 등에서 구조화된 데이터(테이블, 정보)를 쉽게 추출할 수 있습니다. 이는 비정형 문서를 정형 데이터로 변환하는 강력한 도구입니다.
- **최신 모델 지원:** Llama, Gemini, Claude, ChatGPT 등 최신 대규모 언어 모델(LLM)을 데이터브릭스 플랫폼에서 최적화된 GPU 서버를 통해 빠르게 사용할 수 있도록 제공합니다.
- **서버리스 GPU 컴퓨트 및 MCP 서버:** 특정 사용 사례에 대한 추가 가속을 위해 GPU 컴퓨트를 서버리스 형태로 제공하며, AI 에이전트가 상호작용할 수 있는 MCP(Model, Code, Data) 서버를 지원합니다.

0.2.6 데이터 및 AI 거버넌스

데이터브릭스는 데이터의 보안, 품질, 공유를 위한 강력한 거버넌스 도구를 제공합니다.

- **ABAC (Attribute-Based Access Control):**
 - **개념:** 역할(Role)이 아닌 데이터의 속성(Attribute)에 기반하여 접근 권한을 제어하는 방식입니다. 예를 들어, 특정 컬럼이 PII(개인 식별 정보) 태그를 가지고 있다면, 해당 태그에 대한 정책을 자동으로 적용하여 접근을 제한합니다.
 - **장점:** RBAC(Role-Based Access Control)보다 유연하고 확장성이 뛰어납니다. 새로운 데이터셋이 추가되더라도 태그만 올바르게 지정되면 기존 정책이 자동으로 적용되어 관리자의 부담을 줄입니다.
- **마켓플레이스의 MCP 서버 (MCP in the Marketplace):** 조직 내 사용자 정의 데이터, 모델, 코드를 포함하는 MCP 서버를 마켓플레이스에 등록하여 AI 에이전트가 참조하고 활용할 수 있도록 합니다. 이 MCP 서버는 유니티 카탈로그의 거버넌스 하에 관리됩니다.
- **아이스버그 클라이언트와의 공유 (Sharing to Iceberg Clients):** 델타 공유(Delta Sharing) 프로토콜을 통해 델타 테이블뿐만 아니라 아이스버그 형식의 데이터도 공유할 수 있습니다. 유니폼(Uniform) 또는 관리형 아이스버그(Managed Iceberg)를 통해 델타 메타데이터가 생성되어 공유가 가능해집니다.

다.

- **ABAC을 통한 공유 (Sharing with ABAC):** 델타 공유 시 민감한 행이나 컬럼이 있는 경우, ABAC 정책 자체가 수신자 측으로 전달되어 데이터 복사 없이도 정책 기반의 접근 제어가 가능해집니다. 이는 데이터 공유의 보안과 유연성을 크게 향상시킵니다.

0.2.7 데이터 웨어하우징, 마이그레이션 및 엔지니어링

데이터브릭스는 전통적인 데이터베이스 기능을 강화하고, 데이터 마이그레이션 및 엔지니어링 워크플로우를 간소화합니다.

- **멀티 스테이트먼트 트랜잭션 및 저장 프로시저 (Multi-statement Transaction and Stored Procedures):** 관계형 데이터베이스에서 흔히 사용되는 여러 SQL 문장을 하나의 트랜잭션으로 묶어 처리하는 기능과, 복잡한 로직을 캡슐화하는 저장 프로시저를 지원합니다. 이는 데이터 일관성을 보장하고 코드 재사용성을 높입니다.
- **공간 SQL (Spatial SQL):** 지리적 데이터(예: 위도, 경도, 다각형)를 SQL 쿼리로 직접 처리하고 분석할 수 있는 기능을 제공합니다. 재난 지역 분석, 위치 기반 서비스 등 다양한 공간 분석 시나리오에 활용될 수 있습니다.
- **LakeFlow Connect:** 다양한 외부 데이터 소스(파일, 관계형 데이터베이스, 마케팅 클라우드 등)와의 연결을 지원하여 데이터 수집을 간소화합니다. 특히 제로 카피 ETL(Zero-Copy ETL)을 통해 효율적인 데이터 통합을 가능하게 합니다.
- **LakeFlow Designer:** 시각적인 UI를 통해 데이터 파이프라인을 설계할 수 있는 도구입니다. 소스, 필터, 조인, 집계, AI 함수 등 다양한 오퍼레이터를 드래그 앤 드롭 방식으로 구성하면 Spark 선언적 코드(Spark Declarative Pipelines)가 자동으로 생성됩니다.
- **Lakebridge:** 20개 이상의 레거시 데이터 스토어에서 데이터브릭스로 데이터를 마이그레이션하는데 도움을 주는 오픈 소스 도구입니다. 분석기(Analyzer), 변환기(Converter), 데이터 검증기(Data Validator) 기능을 포함하여 마이그레이션 과정을 간소화합니다.

데이터브릭스의 새로운 기능들은 다음 단계를 거쳐 출시됩니다.

- **베타 (Beta):** 초기 개발 단계, 제한된 사용자에게 제공.
 - **프라이빗 프리뷰 (Private Preview):** 일부 고객에게만 접근 권한 부여.
 - **퍼블릭 프리뷰 (Public Preview):** 문서화되어 일반 사용자도 체험 가능.
 - **GA (Generally Available):** 정식 출시, 프로덕션 환경에서 사용 권장.
- 많은 기업들은 GA 단계에서만 프로덕션에 기능을 도입하는 경향이 있습니다.

퀴즈 2 해설

퀴즈 2는 Spark, Delta Lake, 스트리밍, 머신러닝 등 데이터브릭스 플랫폼의 핵심 개념을 평가합니다. 다음은 각 문제에 대한 상세 해설입니다.

0.3 데이터 처리 및 최적화

1. 테이블 생성 시 타임스탬프를 날짜로 변환:

- 문제: 타임스탬프 컬럼을 날짜 컬럼으로 변환하여 테이블을 생성하는 방법.
- 해설: GENERATED ALWAYS AS 절을 사용하여 타임스탬프 컬럼에서 날짜를 자동으로 생성할 수 있습니다. 이는 파티셔닝과 필터링에 유용하며, IDENTITY 절과 함께 사용될 수도 있습니다.
- 예시:

```
1 CREATE TABLE my_table (
2     event_timestamp TIMESTAMP,
3     event_date DATE GENERATED ALWAYS AS (CAST(event_timestamp AS DATE))
4 ) USING DELTA;
```

Listing 3: GENERATED ALWAYS AS 예시

2. Spark 최적화의 4가지 S:

- 문제: Spark 최적화의 일반적인 문제 영역 4가지.
- 해설: 데이터 스큐(Data Skew), 셔플(Shuffles), 스펠(Spills), 스몰 파일(Small Files)입니다. 이 중 일부는 AQE(Adaptive Query Execution)에 의해 자동으로 해결되지만, 복잡한 프로젝트에서는 수동 최적화가 필요할 수 있습니다.

3. Delta Lake의 특징:

- 문제: Delta Lake에 대한 설명 중 옳은 것.
- 해설: Delta Lake는 파티션 프루닝(Partition Pruning) 및 Z-오더링(Z-ordering)과 같은 최적화 기법을 사용합니다. 최근에는 리퀴드 클러스터링(Liquid Clustering)이 이들을 대체하며, 블룸 필터링(Bloom Filtering)도 있습니다. 또한, Delta Lake는 오픈 소스 및 오픈 포맷이며, 높은 확장성을 제공합니다.

4. 데이터 백업 및 시간 여행:

- 문제: 동료가 데이터를 수동으로 백업 폴더에 복사하는 상황에서, 더 나은 방법.
- 해설: Delta Lake의 시간 여행(Time Travel) 기능을 활용하면 모든 트랜잭션이 자동으로 기록되므로, 특정 시점으로 쉽게 되돌아갈 수 있습니다. 수동 복사 없이 데이터 변경 이력을 관리할 수 있습니다.

5. 중복 행 제거:

- 문제: SQL에서 중복 행을 제거하는 방법.
- 해설: `SELECT DISTINCT * FROM tablenname;` .

6. 쿼리 파라미터화:

- 문제: 배치 날짜(batch date)를 입력 파라미터로 사용하여 테이블을 쿼리하는 방법.
- 해설: 노트북 환경에서는 위젯(Widgets)을 사용하여 쿼리를 파라미터화할 수 있습니다. 순수 SQL에서는 SQL 변수나 명명된/명명되지 않은 파라미터를 사용할 수 있습니다.

7. Spark의 AQE (Adaptive Query Execution):

- **문제:** Spark의 AQE가 무엇인지.
- **해설:** AQE는 Spark SQL 쿼리 실행 계획을 런타임에 동적으로 조정하여 성능을 최적화하는 쿼리 최적화 도구입니다.

8. Lakehouse가 데이터 레이크 및 데이터 웨어하우스 의존성을 대체하는 방법:

- **문제:** Lakehouse가 기존 데이터 레이크 및 데이터 웨어하우스의 의존성을 대체하는 방법.
- **해설:** Lakehouse는 배치 및 스트리밍 분석을 모두 가능하게 하며, 스토리지와 컴퓨트를 분리하고, Parquet과 같은 표준 데이터 형식을 사용하며, 다양한 API를 지원합니다. 따라서 모든 보기가 정답입니다.

9. 특정 컬럼에 대한 중복 행 제거:

- **문제:** `product_id`.
- **해설:** `dropDuplicates()` 메서드를 사용하고, 중복을 확인할 컬럼 이름을 리스트 형태로 전달합니다.
- **예시:**

```
1 df.dropDuplicates(["product_id"])
```

Listing 4: dropDuplicates 예시

10. SQL MERGE INTO 명령어:

- **문제:** 조건에 따라 행을 삽입, 업데이트, 삭제하는 SQL 명령어.
- **해설:** MERGE INTO는 단일 명령으로 조건부 삽입, 업데이트, 삭제를 수행할 수 있는 강력한 SQL 구문입니다.

11. 컬럼 값의 최대/최소값 표시:

- **문제:** `product_id value`.
- **해설:** `groupBy()`와 `agg()` 메서드를 사용하여 `product_id min() max()`.
- **예시:**

```
1 from pyspark.sql.functions import min, max
2 df.groupBy("product_id").agg(min("value").alias("min_value"), max("value")
    ).alias("max_value"))
```

Listing 5: groupBy 및 agg 예시

12. 모델 생성의 가장 효과적인 형태:

- **문제:** 모델 생성의 가장 효과적인 형태.
- **해설:** 사용 사례(use case)에 따라 다릅니다. 특정 모델이나 방법론이 항상 최고라고 할 수는 없습니다.

0.4 스트리밍 처리

1. Delta 테이블 위에 스트리밍 뷰 생성:

- **문제:** Delta 테이블 위에 스트리밍 뷰를 생성하는 SQL 문.
- **해설:** `readStream` 옵션을 사용하여 Delta 테이블을 스트리밍 소스로 읽고, `trigger(availableNow=True)`와 같은 트리거 옵션을 지정하여 스트리밍 쿼리를 실행합니다.
- **예시:**

```
1 CREATE OR REPLACE TEMPORARY VIEW sales_stream_view AS
```

```

2 SELECT * FROM read_files(
3   'path/to/delta_table',
4   format => 'delta',
5   read_options => map('maxFilesPerTrigger', '1')
6 );

```

Listing 6: 스트리밍 뷰 생성 예시

(강의에서는 `CREATE OR REPLACE TEMP VIEW sales_stream AS SELECT * FROM STREAM(table_name)`)

2. 구조화된 스트리밍의 내결함성 (Fault Tolerance):

- 문제: 구조화된 스트리밍이 중단 간 내결함성을 위해 사용하는 기술.
- 해설: 체크포인팅(Checkpointing)과 아이덴포턴트 싱크(Idempotent Sinks)의 조합을 통해 "정확히 한 번(Exactly-Once)" 처리를 보장하여 내결함성을 확보합니다.

0.5 Spark 고급 기능

1. 브로드캐스트 변수 (Broadcast Variables):

- 문제: 브로드캐스트 변수에 대한 설명 중 옳은 것.
- 해설: 브로드캐스트 변수는 불변(immutable)하며, 클러스터 전체에 공유되지 않고 각 워커 노드에 로컬로 복사됩니다.

2. 셔플 (Shuffle)의 정의:

- 문제: 셔플에 대한 설명.
- 해설: 셔플은 Spark에서 데이터를 클러스터 전체에 재분배하여 병렬 처리를 가능하게 하는 과정입니다. 이는 성능 병목의 주요 원인이 될 수 있습니다.

3. 브로드캐스트 조인 (Broadcast Join)의 유효성:

- 문제: 큰 데이터프레임과 작은 데이터프레임을 $item_i d$, broadcast .
- 해설: broadcast는 유효한 조인 타입이 아닙니다. broadcast() 함수는 작은 데이터프레임을 브로드캐스트하여 조인 성능을 향상시키는 데 사용됩니다.
- 예시: `df_large.join(broadcast(df_small), "item_id")`

4. 파티션 수 변경:

- 문제: 데이터프레임의 파티션 수를 12개에서 24개로 효율적으로 변경하는 코드 블록.
- 해설: 파티션 수를 늘릴 때는 `repartition()`을 사용합니다. `coalesce()`는 파티션 수를 줄일 때 사용되며 오버헤드가 적습니다.
- 예시:

```

1 df.repartition(24)

```

Listing 7: repartition 예시

5. 배열 컬럼 폭발 (Explode):

- 문제: 배열 타입의 attribute 컬럼을 개별 행으로 폭발시키는 방법.
- 해설: explode() 함수는 select() 문 내에서 사용되어 배열의 각 요소를 새로운 행으로 변환합니다. explode는 데이터프레임의 메서드가 아닙니다.
- 예시:

```

1 from pyspark.sql.functions import explode

```